Static Code Analysis

Digital Wallet & Payment Tracker

SRN: PES1UG24CS578 | Submission: 16 October 2026

Objective: improve the assigned Python program's quality, security and style using Pylint, Bandit and Flake8, then verify the changes.

Verified results

Check	Original	Cleaned submission
Pylint	5.69/10; 25 messages	10.00/10; 0 messages
Bandit	4 findings: 1 high, 1 medium, 2 low	0 findings; no skipped checks
Flake8	4 findings	0 findings; exit code 0
Regression tests	No tests in assigned source	18 tests passed
Demo balance / total credits	Rs. 1900.0 / Rs. 2500	Rs. 1900.0 / Rs. 2500

Method and source

Cloned SETAPESU26/L5_29_digital_wallet at commit 0d188e2b4c3588ba9e3e3f7362f426d10beb7d43. The original source and README are preserved unchanged under original/. Ran the demo and all three tools before editing, prioritized the high and medium Bandit findings, fixed quality and formatting defects, then reran the checks.

Submission contents

The cleaned digital_wallet.py, issues.md, reflection.md, six before/after tool reports, original and final demo outputs, test evidence, tool versions and three tool-specific commits are saved under LAB5/. The top-level pylint_report.txt, bandit_report.txt and flake8_report.txt are copies of the final reports. An empty Flake8 report means no findings.

Environment

Python 3.12.14; Pylint 4.1.3; Bandit 1.9.4; Flake8 7.4.1. Tools ran in a dedicated virtual environment. Full dependency versions and raw exit codes are retained with the reports.

Issue table / security and quality

All line references are to original/digital_wallet.py. Bandit severities are copied from the actual report; other priorities are lab assessments.

Issue / type	Lines	Description	Fix approach
B324 - weak PIN hash / Bandit: High	14	SHA-1 hashes the PIN directly without salt or a work factor.	Replace with PBKDF2-HMAC-SHA256, 600,000 iterations, a fresh 16-byte random salt and a stored in-memory verifier. Compare verifiers with `hmac.compare_digest`.
B306 - unsafe receipt filename / Bandit: Medium	60	`tempfile.mktemp` returns a name without atomically creating a file, allowing a race.	Use `mkstemp`, close the returned descriptor, and document caller cleanup. Test that distinct empty files actually exist.
B105 - embedded gateway key / Bandit: Low	6	A gateway secret is embedded in source.	Configure credentials explicitly. The demo reads environment values or generates ephemeral random credentials; remove the literal key. Also remove the embedded PIN at line 7 and successful-payment PIN literals at 70-73 as related manual hardening.
B311 - predictable OTP source / Bandit: Low	64	`random.randint` is not intended for security tokens.	Use `secrets.randbelow(900000) + 100000`; the OTP remains six digits.
W0102 - mutable default argument / Pylint: correctness priority	17, 23	The same default log list is reused on calls that omit the argument.	Use `log=None` and create a fresh list inside the function; keep caller-provided logs working.
W0603 - global rebinding / Pylint: maintainability priority	28-29	Updating the limit rebinds a module global.	Store balance and limit in the explicit `WALLET` state dictionary, so the function needs no `global` statement.
W0613 - unused category argument / Pylint: maintainability priority	50	`merchantType` is accepted but ignored.	Rename it to `merchant_type` and validate that it is a nonempty category; preserve the documented flat 2% rate for every category.

Issue table / conventions and style

Issue / type	Lines	Description	Fix approach
C0103 - naming conventions / Pylint: convention	9, 13, 17, 27, 32, 45, 50, 54, 58, 78	Mixed camelCase names and a lowercase limit constant reduce consistency.	Use snake_case function/argument/local names and uppercase names for module state containers and constants.
C0114 / C0116 - missing documentation / Pylint: convention	1, 13, 17, 27, 32, 45, 50, 54, 58, 63	The module and its functions have no docstrings.	Document the demo, state model, validation, credential handling and receipt cleanup.
E261 - inline comment spacing / Flake8: style	76	Only one space precedes the inline comment.	Insert the required two spaces.
E225 - assignment spacing / Flake8: style	78	Assignment has no surrounding spaces.	Format the assignment with spaces and wrap the comprehension.
E231 - comma spacing / Flake8: style	79	The print call omits a space after a comma.	Add the required space.
E305 - missing blank lines / Flake8: style	80	A module-level call immediately follows the function definition.	Separate top-level blocks with two blank lines and use an `if __name__ == "__main__"` guard.

Additional review findings

These four defects were found by inspection and tested separately. They are not counted as tool findings.

Issue / type	Lines	Description	Fix approach
Negative/nonfinite payments / Manual correctness review	35-39	A negative debit increases the balance; NaN can bypass comparisons and poison state.	Reject zero, negative, nonfinite, boolean and nonnumeric payment amounts before authentication or mutation. Apply the same validation to credits and limits.
Invalid limit / Manual correctness review	27-29	Any value can replace the cap.	Require a positive finite number and raise `ValueError` before changing the previous cap.
Zero statement count / Manual correctness review	45-46	`transactions[-0:]` prints the entire history.	Return without output for count 0 and reject invalid counts.
Import side effects / Manual maintainability review	80	Importing the module runs payments and prints an OTP.	Guard the demo entry point; reset state when explicitly rerunning the demo.

Reflection / tools and priorities

The supplied handout contains no explicit reflection prompts. The following four relevant questions were selected from the learning objectives with the user's approval.

1. What did each tool contribute, and why use all three?

Pylint found code-quality and maintainability issues: the shared default log, global rebinding, unused category argument, inconsistent names and missing docstrings. Bandit highlighted security-sensitive APIs and data: SHA-1 PIN hashing, a hardcoded gateway key, an unsafe temporary filename and an OTP generated with `random`. Flake8 found spacing and top-level layout defects. Their scopes overlap but complement one another. A good formatting result would not have shown that the PIN hashing or temporary-file handling was unsafe, while a clean Bandit run would not have fixed the shared list.

2. How were findings prioritized, and what did the fixes change?

The first priority was the high-severity B324 PIN hash finding, followed by medium-severity B306 temporary-file creation. I replaced direct SHA-1 with salted PBKDF2-HMAC-SHA256, and `mktemp` with atomic `mkstemp` creation. I then addressed both low-severity Bandit findings by generating OTPs with `secrets` and moving credentials out of source. The remaining work corrected the mutable log default and other Pylint issues before normalizing Flake8 formatting. Bandit's severity was used as reported; the low rating of the embedded demo key does not mean real embedded credentials are acceptable.

Examples to explain during presentation

A shared log=[] default survives between function calls, so independent callers can share history accidentally. Using None allocates a new list when needed. A filename returned by mktemp is not reserved; mkstemp creates and opens the file atomically. PBKDF2 adds a salt and repeated work, while secrets supplies randomness suitable for security tokens.

Reflection / verification and limits

3. How was correctness checked, and what did static analysis miss?

I saved the original output and all three reports before changing the code. Afterwards, Pylint improved from 5.69/10 to 10.00/10, all four Bandit findings were removed and Flake8 produced no findings. Eighteen regression tests checked the original demonstration, payment rejection, limits, credential validation, isolated logs, receipt-file creation and import behavior. The demo still prints the same results after the randomly generated OTP line. Manual inspection also found negative-debit and NaN bugs that the tools did not report. Testing those boundaries showed why static analysis must be combined with code review and execution tests.

4. How does this improve maintainability, and what are the limits?

Clear names and docstrings make the wallet easier to explain and modify. Explicit state and credential setup make behavior more predictable, while tests catch regressions when the implementation changes. I would run these tools before commits or in continuous integration and review new findings instead of suppressing them just to improve a score. A clean report remains limited to the rules and paths analyzed: this demo still uses in-memory state, a per-payment cap rather than cumulative daily accounting, numeric money values inherited from the source and console OTP output. Production work would require persistent transactions, exact currency accounting, rate-limited authentication and private OTP delivery. The lab establishes a repeatable quality-checking process rather than proof of complete security.

Reproduce and inspect the evidence

Use README.md for setup and commands. Analyze digital_wallet.py explicitly; a folder-wide scan also includes the intentionally unfixed source in original/. reports/after/run_metadata.json records exit code 0 for all three tools, the demo and the test suite. No rule suppression was used.

Primary references

Python documentation: docs.python.org/3/library/hashlib.html (key derivation); docs.python.org/3/library/secrets.html (token generation); docs.python.org/3/library/tempfile.html (mkstemp). Full clickable references and rule IDs are in issues.md.